faultbox: Cost, Coverage and Recovery of In-Process Fault Isolation with Memory Protection Keys

Meghashyam Sai Dontha, Blekinge Institute of Technology (BTH)

November 2026

Code and data: https://github.com/saishyamdontha/faultbox

Abstract

Systems that run untrusted code inside a host process depend on components outside any sandbox, and a fault in one of them can crash or corrupt the whole process. This report compares seven ways for a C host to call untrusted plugins: no isolation, three forms of process isolation (a sandboxed process per call, a persistent sandboxed worker over pipes, and one over shared memory), and three variants of in-process isolation using x86 memory protection keys (MPK/PKU) with signal-based recovery. Against nine faulty or malicious plugins, process isolation keeps the host intact in all nine cases. Plain MPK isolation manages six, and a hardened variant with a seccomp-BPF exit gate and a `WRPKRU` scanner manages eight. The clearest advantage of in-process isolation is recovery: restoring the plugin's memory from a snapshot takes 1.5 µs, about 80x faster than respawning a worker. Per call, MPK adds about 21 ns, about half the overhead of the fastest process boundary measured (40 ns), which in turn occupies a dedicated core. A naive MPK implementation costs 301 ns per call, about 92% of it from one signal-mask syscall per call.

1. Motivation

Process isolation is the standard way to contain untrusted code, but every call crosses a process boundary, and every crash requires a new process. In-process isolation divides one address space into memory domains instead. MPK supports this in hardware: each page carries a 4-bit key, and a per-thread register (PKRU), changed with the unprivileged `WRPKRU` instruction, controls access to each key. This project asks:

1. Which faults does each strategy contain?
2. What does each cost per call and per recovery?
3. Where does MPK-based isolation stop working as a boundary?

2. Threat and fault model

The host owns `host_state`, one page tagged with a protection key in MPK modes, and `host_config`, an ordinary global that is never protected. Plugins are shared objects loaded with `dlopen`.

Plugin	Behaviour	Class
`nullderef`	Null pointer dereference	crash (SIGSEGV)
`oob_write`	Writes into `host_state`	corruption (protected)
`corrupt_unprotected`	Writes into `host_config`	corruption (unprotected)
`divzero` / `illegal` / `abort_call`	Division by zero / `ud2` / `abort()`	crash (SIGFPE / SIGILL / SIGABRT)
`stack_overflow`	Unbounded recursion	crash (guard page)
`exit_call`	Calls `exit(42)`	process termination
`wrpkru_attack`	Sets PKRU = 0 with `WRPKRU`, writes `host_state`	malicious bypass

A control plugin (`good`) and a stateful plugin (for recovery correctness) complete the set. The primary model is a buggy plugin; the malicious plugin tests where the boundary fails.

3. Design

Mode	Isolation	Recovery
`none`	None	None
`fork`	New process per call, `seccomp` strict	New process per call
`ipc`	Persistent worker, `seccomp` strict, pipes	Respawn worker
`ipc_shm`	Persistent worker, `seccomp` strict, shared memory, spinning on a second core	Respawn worker
`mpk`	Protection key disabled during plugin calls; signal handler recovers	Reload plugin
`mpk_fast`	As `mpk`, without saving the signal mask	Reload plugin
`mpk_hard`	As `mpk_fast`, plus seccomp-BPF exit gate and `WRPKRU` scan at load	Restore memory snapshot

Key implementation details:

- **Signals and PKRU.** MPK modes handle SIGSEGV, SIGBUS, SIGFPE, SIGILL, SIGABRT and SIGSYS on an alternate signal stack, so stack overflows are recoverable. The kernel resets PKRU on signal delivery and `siglongjmp` skips `sigreturn`, so the host restores access to its key explicitly.
- **Signal mask.** `sigsetjmp(buf, 1)` saves the mask with a syscall on every call. Skipping it is safe only with an `SA_NODEFER` handler.
- **Exit gate.** Seccomp filters are permanent, so `exit` and `exit_group` are blocked for the whole process unless issued from one instruction in the host, identified by the instruction pointer in the filter. A plugin calling `exit()` receives SIGSYS and is recovered.
- **Snapshot recovery.** At load, the plugin's writable segments (excluding RELRO) are copied; after a fault they are copied back.
- **Worker respawn.** A new `ipc_shm` worker signals readiness before the host spins. Without this, the worker starved on the host's core and recovery took about 3 ms instead of 119 µs.

4. Evaluation setup

AMD Ryzen 9 8940HX (PKU supported), Ubuntu 24.04, gcc; host `-O2`, plugins `-O0`. Performance power profile, on AC power. Containment: each (mode, plugin) pair in a fresh process, classified from the exit status and checks of both memory regions. Per-call cost: calls to `good` after 1,000 warm-up calls (1,000,000 calls for `none`, MPK modes and `ipc_shm`; 100,000 for `ipc`; 10,000 for `fork`). Recovery: mean over 1,000 crashing calls, including the recovery step. Pinned to CPU 2 (`ipc_shm`: CPUs 2 and 4, separate physical cores). 5 repetitions, means reported.

5. Results

5.1 Containment

Plugin	none	fork / ipc / ipc_shm	mpk, mpk_fast	mpk_hard
nullderef, divzero, illegal, abort_call, stack_overflow	crashed	contained	contained	contained

Plugin	none	fork / ipc / ipc_shm	mpk, mpk_fast	mpk_hard
oob_write	corrupted	ok*	contained	contained
corrupt_unprotected	corrupted	ok*	**corrupted**	**corrupted**
exit_call	host exited	contained	**host exited**	contained
wrpkru_attack	corrupted	ok*	**corrupted**	rejected at load
Host intact (of 9)	**0**	**9**	**6**	**8**

* The write landed in the worker's copy-on-write memory.

Only `oob_write` is stopped by the protection key itself (`SEGV_PKUERR`); the other crash faults are survived because of the signal handler and the alternate stack.

5.2 Performance

Mode	Per call	Overhead	Fault + recovery
none	2.6 ns	–	–
mpk_hard	23.3 ns	+20.7 ns	1.5 µs
mpk_fast	23.7 ns	+21.1 ns	29.2 µs
mpk	301 ns	+299 ns	30.2 µs
ipc_shm	42.2 ns	+39.6 ns	119.0 µs
ipc	5,394 ns	+5.4 µs	117.5 µs
fork	106,883 ns	+107 µs	107.4 µs

5.3 Recovery correctness

A stateful plugin returns 1, scrambles its state to 12345, and crashes. Without recovery (negative control) the next call returns 12346; with reload or snapshot restore it returns 1. All recovery methods passed, and the control failed as expected.

6. Discussion

Recovery, not call speed, is the main advantage. Against a well-engineered process boundary, MPK's per-call overhead is only about half (21 vs 40 ns), and the process boundary pays for its speed with a permanently busy core. Recovery differs by nearly two orders of magnitude: 1.5 µs for a snapshot restore against about 118 µs for any worker respawn.

Recovery design matters as much as the isolation mechanism. With the same isolation and the same fault, reloading the plugin takes 29 µs and restoring a snapshot 1.5 µs.

Bookkeeping can dominate. About 92% of naive MPK overhead was one `rt_sigprocmask` syscall per call (101,000 calls under `strace`, against 0 for the optimised version).

Coverage needs more than MPK. MPK alone protects only tagged memory and does not restrict syscalls. A seccomp exit gate and a load-time scan close two of the three remaining gaps; protecting all host memory would require restructuring the host into domains.

Implication for runtimes hosting untrusted code. WebAssembly engines and similar runtimes already sandbox guest code, but host functions, system-interface implementations and third-party libraries run outside the sandbox. These results suggest that in-process domains around such components are cheap per call, and that their strongest benefit is fast, fine-grained recovery without restarting the runtime. The hard parts are coverage, syscall restriction and correct state reset.

7. Limitations and future work

Only one page is key-protected. Exit-time side effects run before the exit gate stops a plugin. The `WRPKRU` scanner misses library calls, `XRSTOR` and generated code, and has a time-of-check/time-of-use window. Snapshots exclude heap, mmaps and TLS, and a fault inside a libc lock could deadlock any in-process recovery. Single-threaded, no hang detection, one machine.

Future work: per-plugin keys for co-located plugins; heap-aware snapshots; scanning through the opened file descriptor; Intel CPUs; multi-threaded hosts.

8. Conclusion

On this machine, MPK-based isolation with a seccomp exit gate contained 8 of 9 faulty or malicious plugins at about 21 ns per call, and recovered from crashes in 1.5 µs, about 80x faster than respawning a sandboxed worker. Per call, it was about twice as cheap as the fastest process boundary measured. Its remaining weakness is structural: it protects only memory explicitly placed in a protected domain.

References

1. A. Vahldiek-Oberwagner et al., "ERIM: Secure, Efficient In-process Isolation with Protection Keys (MPK)," USENIX Security Symposium, 2019.
2. M. Hedayati et al., "Hodor: Intra-Process Isolation for High-Throughput Data Plane Libraries," USENIX Annual Technical Conference, 2019.
3. M. Gülmez et al., "Rewind & Discard: Improving Software Resilience using Isolated Domains," IEEE/IFIP DSN, 2023.
4. Linux man-pages: `pkeys(7)`, `seccomp(2)`, `sigaction(2)`, `sigaltstack(2)`, `sigsetjmp(3)`.